

AI Fill — Section & Field Granular Generation

SECTION autofill (✨ on section header)

1. User clicks ✨ on section header
2. CharacterSection calls handleAiFillSection(sectionId)
3. Check: if aiSectionLoading → return (mutex)
4. POST /api/ai/fill with { sectionIds: [sectionId], stream: true }
5. Server: buildFillPrompt filters schema to sections fields only
6. SSE → parsePartialJson → setData incrementally
7. Stream ends → doSave → add section to openSections

Key difference from full fill:

- Only sections fields generated
- Other sections keep existing data
- openSections adds (not replaces)

FIELD autofill (magic_button on each field)

1. User clicks magic_button on field
2. CharacterSection calls handleAiFillField(fieldId)
3. Check: if aiFieldLoading || aiSectionLoading || aiLoading → return
4. POST /api/ai/fill with { fieldIds: [fieldId], stream: true }
5. Server: buildFillPrompt generates prompt for single field
6. SSE → parsePartialJson → setData (only 1 field)
7. Stream ends → doSave → highlight field (5s animation)

Key constraints:

- Mutually exclusive: field blocks during section fill
- Abort button replaces magic_button during loading

SHARED post-processing (all 3 levels: full / section / field):

- All go through the same /api/ai/fill endpoint
- All use SSE streaming with fetchSseStream + parsePartialJson
- First chunk triggers pushUndo (for Ctrl+Z)
- Stream ends → doSave (800ms debounce) → SQLite via Server Action
- FixedFields animation (5s highlight on changed fields)
- Error → setAiError; Abort → setAiError('Request cancelled')